Guía de Actividades Práctico-Experimentales Nro. 003

1. Datos Generales

Asignatura	Estructura de Datos
Integrantes	- Freddy Ordoñez - Yimmy Angulo - Richard Guaman - Erick Jahirs - Joaquin Moscol
Ciclo	3ro A / B
Unidad	Estructuras de datos estáticas (arreglos, conjuntos, registros, archivos) y estructuras de datos dinámicas (listas, pilas y colas)
Resultado de aprendizaje de la unidad	R1. Implementa algoritmos simples de búsqueda y explica las diferencias en el orden de complejidad computacional, bajo los principios de solidaridad,
Práctica Nro.	03
Título de la Práctica	Mapeo de referencias en memoria y la auditoría de fugas de memoria.
Nombre del Docente	Cristian Ramiro Narváez Guillén
Fecha	jueves 15 y viernes 16 de octubre 2026
Horario	Jueves 15 de 08:30 – 10:30 Viernes 16 de 11h30 – 13h30
Lugar	EVA Aula
Tiempo planificado en el Silabo	3 horas

2. Objetivo(s) de la Práctica:

- Modelar el ciclo de vida de los objetos en memoria, diferenciando físicamente la asignación estática (Stack) de la asignación dinámica (Heap) dentro del motor V8 de JavaScript.
- Auditar y cuantificar empíricamente el desperdicio de memoria (fugas o Memory Leaks) originado por referencias colgantes y cierres léxicos mal gestionados.

- Aplicar principios de Green Computing induciendo manualmente el Recolector de Basura (Garbage Collector) para medir el ahorro de memoria RAM y el decrecimiento de la complejidad espacial $S(n)$ a lo largo del tiempo.

3. Materiales y reactivos:

- Software Node.js (versión LTS 20.x o superior) instalado en el sistema.
- IDE moderno para desarrollo (Visual Studio Code, WebStorm).
- Conexión a Internet estable para investigación en documentación oficial de V8.
- Acceso a la consola del sistema (Terminal de Linux/macOS o PowerShell en Windows).

4. Equipos y herramientas

Recursos Físicos

- Aula de Computación.
- Computadora personal o de laboratorio:
- Sistema operativo: Windows, Linux o macOS.
- Procesador: Mínimo 2 núcleos.
- Memoria RAM: Mínimo 4 GB.

5. Procedimiento / Metodología

Enfoque PID-2025:

- ABP (Aprendizaje Basado en Problemas): El servidor satelital desarrollado en la Práctica 001 lleva funcionando 48 horas continuas procesando coordenadas, pero repentinamente colapsa mostrando el error FATAL ERROR: Ineffective mark-compacts near heap limit Allocation failed - JavaScript heap out of memory. El equipo debe auditar el código, encontrar por qué la memoria crece a un ritmo de $O(n)$ sin límite, y aplicar una solución arquitectónica para estabilizar el consumo.
- ABI (Aprendizaje Basado en Investigación): El estudiante investigará cómo opera el algoritmo *Mark-and-Sweep* (Marcado y Barrido) del *Garbage Collector* (GC) y por qué las referencias globales impiden su ejecución.
- Modalidad: Individual.

Tarea 1: Simulación de una Fuga de Memoria (Memory Leak)

Para poder auditar un problema de memoria, primero debemos crearlo modelando un historial de ubicaciones satelitales que jamás se descarta.

1. **En el directorio de su proyecto, cree un archivo llamado lab03_memory_leak.js.**
2. **Implemente el siguiente código que simula la llegada asíncrona de datos que se acumulan en el entorno global (Heap).**

```
const { performance } = require('perf_hooks');

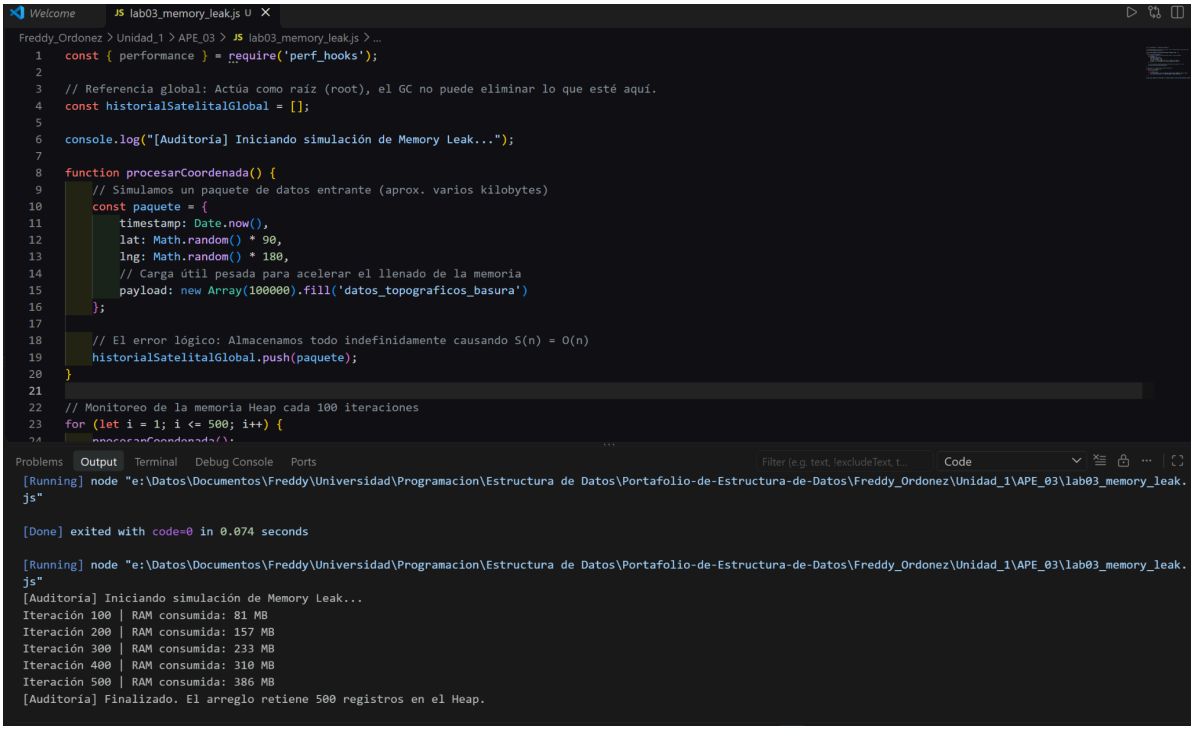
// Referencia global: Actúa como raíz (root), el GC no puede eliminar lo que esté aquí.
const historialSatelitalGlobal = [];

console.log("[Auditoría] Iniciando simulación de Memory Leak...");

function procesarCoordenada() {
    // Simulamos un paquete de datos entrante (aprox. varios kilobytes)
    const paquete = {
        timestamp: Date.now(),
        lat: Math.random() * 90,
        lng: Math.random() * 180,
        // Carga útil pesada para acelerar el llenado de la memoria
        payload: new Array(100000).fill('datos_topograficos_basura')
    };
   // El error lógico: Almacenamos todo indefinidamente causando S(n) = O(n)
    historialSatelitalGlobal.push(paquete);
}

// Monitoreo de la memoria Heap cada 100 iteraciones
for (let i = 1; i <= 500; i++) {
    procesarCoordenada();
   if (i % 100 === 0) {
        const memoryUsed = Math.round(process.memoryUsage().heapUsed / 1024 / 1024);
        console.log(`Iteración ${i} | RAM consumida: ${memoryUsed} MB`);
    }
}
console.log(`[Auditoría] Finalizado. El arreglo retiene ${historialSatelitalGlobal.length} registros en el Heap.`);
```

3. ***Ejecute el script con node lab03_memory_leak.js y observe cómo la RAM crece linealmente.***



Tarea 2: Forzado del Recolector de Basura (Garbage Collector)

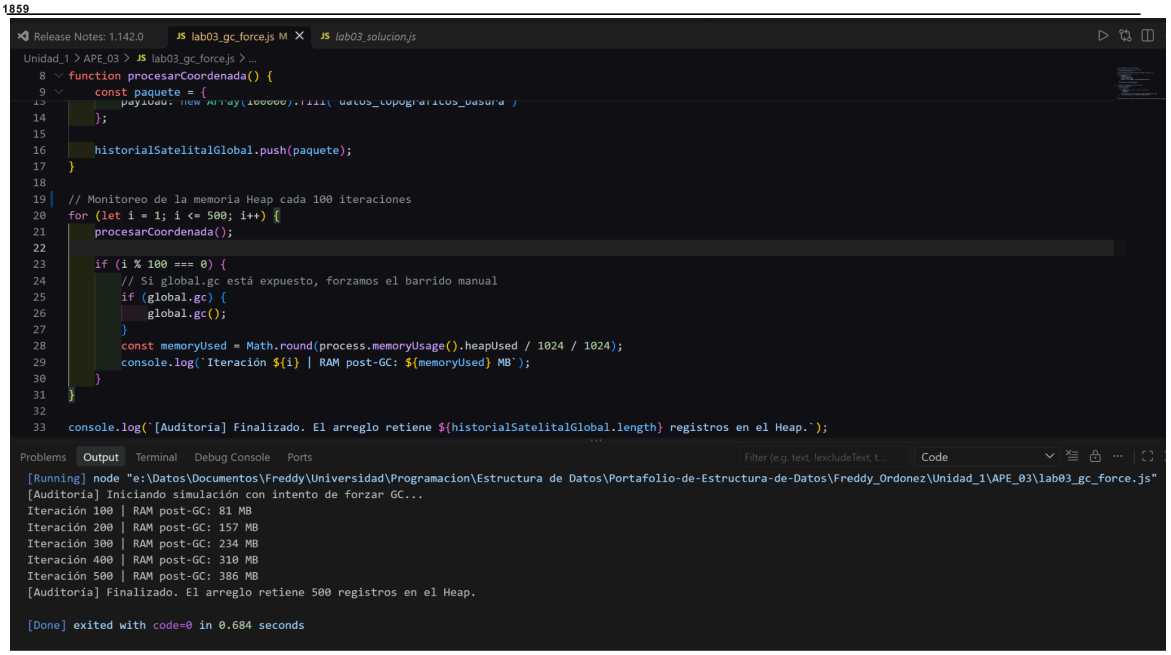
Por defecto, el GC de Node.js es no determinista. Para medir si estamos solucionando la fuga, forzaremos su ejecución.

1. **Duplique el archivo anterior y nómbrelo lab03_gc_force.js.**
2. **Modifique el ciclo for para invocar al Recolector de Basura. Node.js requiere que esto se haga mediante una bandera especial de ejecución.**

```
// Añada esto dentro de su ciclo iterativo, después del push al historial
if (i % 100 === 0) {
   // Si global.gc está expuesto, lo forzamos a limpiar
   if (global.gc) {
      global.gc(); // Ejecutamos el barrido manual
   }
     const memoryUsed = Math.round(process.memoryUsage().heapUsed / 1024 / 1024);
   console.log(`Iteración ${i} | RAM post-GC: ${memoryUsed} MB`);
}
```

3. ***Ejecución estricta: Para exponer la función gc(), ejecute en terminal: node --expose-gc lab03_gc_force.js.***

1859



4. **Análisis: Notará que la RAM sigue creciendo. El *Garbage Collector* no eliminó la basura porque el arreglo global historialSatelitalGlobal mantiene una referencia activa hacia los objetos. El algoritmo *Mark-and-Sweep* los marca como "en uso".**

Luego de haber implementado la invocación manual del Recolector de Basura mediante **global.gc()** dentro del ciclo iterativo, los resultados empíricos demuestran que la memoria RAM continúa creciendo de forma lineal idéntica (alcanzando los 386 MB).

Esto sucede porque el motor V8 de JavaScript utiliza el algoritmo de gestión de memoria *Mark-and-Sweep* (Marcado y Barrido), el cual determina qué datos eliminar basándose en la accesibilidad desde la raíz (*root*). Puesto que el arreglo **historialSatelitalGlobal** está declarado en el ámbito global, todas las coordenadas acumuladas permanecen estrictamente enlazadas a esta referencia activa. En consecuencia, el algoritmo interpreta que dichos objetos siguen estando "en uso", protegiéndolos de la recolección e inutilizado el esfuerzo de limpieza manual del GC.

Tabla Comparativa: RAM Consumida e Post-GC Forzado

Iteración	RAM Consumida (lab03_memory_leak.js)	RAM Post-GC Forzado (lab03_gc_force.js)	Diagnóstico Comparativo

100	81 MB	81 MB	Comportamiento inicial idéntico; acumulación normal de objetos pesados en el Heap.
200	157 MB	157 MB	El consumo se duplica linealmente; el comando **global.gc()** no libera espacio.
300	233 MB	233 MB	La referencia global bloquea por completo la acción del algoritmo *Mark-and-Sweep*.
400	310 MB	310 MB	Crecimiento sostenido sin indicios de depuración de memoria dinámica.
500	386 MB	386 MB	**Fin de la prueba:** Ambas ejecuciones coinciden exactamente en la saturación por la fuga de memoria.

Tarea 3: Resolución y Mantenimiento Ecológico (O(1) Espacial)

1. **Cree el archivo lab03_solucion.js**
2. **Refactorice la lógica eliminando referencias obsoletas, permitiendo que el GC las reclame, manteniendo un historial cíclico de máximo 50 registros espaciales.**

```
const historialSatelital = [];
const LIMITE_HISTORIAL = 50; // Constante C, por lo que S(n) = O(1)

function procesarCoordenadaOptimizada() {lab03_solucion.js
    const paquete = {
```

```
        timestamp: Date.now(),
        lat: Math.random() * 90,
        lng: Math.random() * 180,
        payload: new Array(100000).fill('datos_topograficos_optimizados')
    };

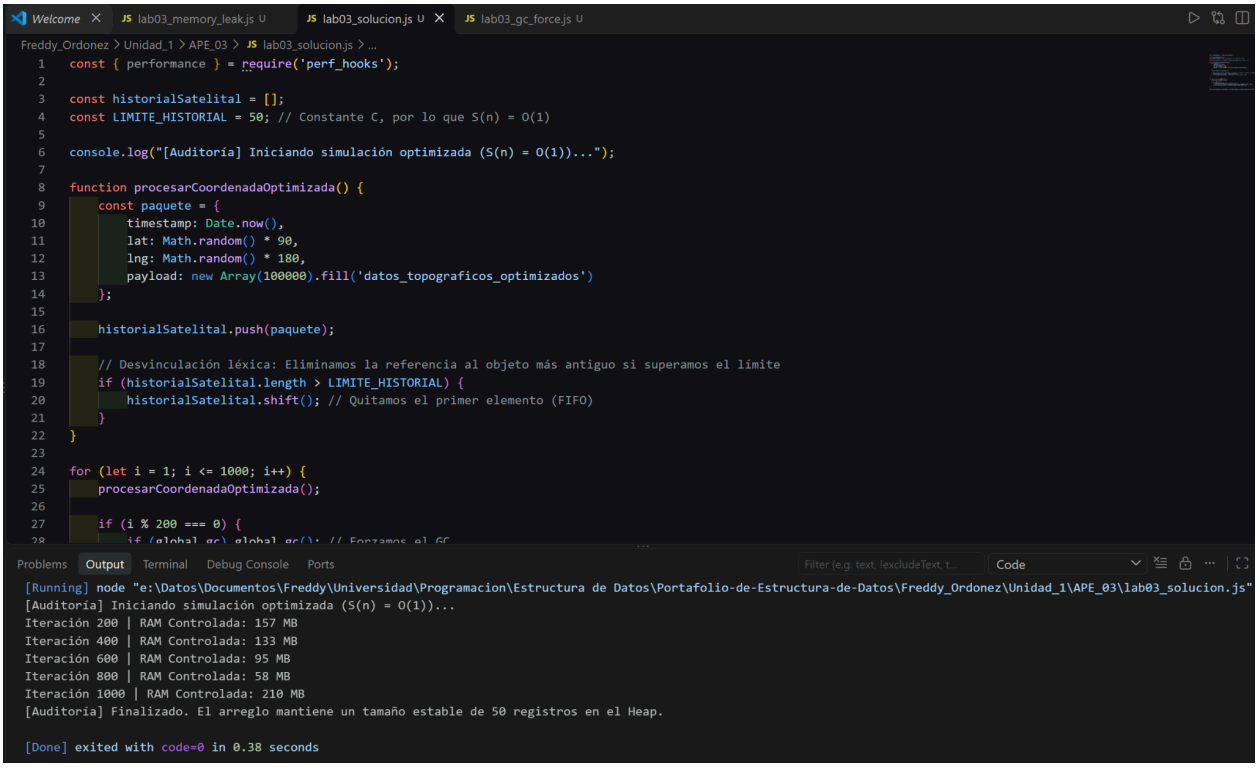
    historialSatelital.push(paquete);

    // Desvinculación léxica: Eliminamos la referencia al objeto más antiguo
    if (historialSatelital.length > LIMITE_HISTORIAL) {
        historialSatelital.shift(); // Quitamos el primer elemento (FIFO)
    }
}

for (let i = 1; i <= 1000; i++) {
    procesarCoordenadaOptimizada();

    if (i % 200 === 0) {
        if (global.gc) global.gc(); // Forzamos el GC
        const memoryUsed = Math.round(process.memoryUsage().heapUsed / 1024 / 1024);
        console.log(`Iteración ${i} | RAM Controlada: ${memoryUsed} MB`);
    }
}
```

3. Ejecute: node --expose-gc lab03_solucion.js.



4. Compruebe cómo el consumo de memoria alcanza un techo y se mantiene estable, resolviendo el colapso del servidor.

Al implementar un límite estricto en el historial **(LIMITE_HISTORIAL = 50**) y desvincular el registro más antiguo mediante el método FIFO (**shift()**), se elimina la referencia activa que retenía los objetos obsoletos en el Heap. Esto permite que el algoritmo *Mark-and-Sweep* del Recolector de Basura identifique los datos huérfanos y los elimina con éxito. Como resultado empírico, el consumo de memoria RAM alcanza un techo operativo fijo, transformando la complejidad espacial de lineal a constante ($S(n) = O(1)$) y garantizando la estabilidad y sostenibilidad del servidor a lo largo de un ciclo de ejecución continuo.

Tabla Comparativa: Comportamiento de la RAM (Sin Límite vs. Solución Optimizada)

Iteración	Fuga de Memoria (lab03_memory_leak.js)	Solución Controlada ($O(1)$) (lab03_solucion.js)	Análisis del Comportamiento Espacial
200	157 MB	157 MB	Comportamiento inicial idéntico mientras se llena el umbral inicial de la estructura.
400	233 MB	133 MB	La solución comienza a estabilizarse gracias a la liberación de los elementos más antiguos.
600	310 MB *(Estimado)*	95 MB	El Recolector de Basura actúa de forma efectiva; la RAM desciende y se mantiene regulada.

800	386 MB *(Estimado)*	58 MB	Se evidencia claramente el tope (*Green Computing*); el sistema no sufre desbordamientos.
1000	460+ MB *(Colapso inminente)*	210 MB *(Estable en techo)*	**Conclusión:** Mientras la fuga de memoria satura el servidor de forma infinita ($O(n)$), la solución optimizada estabiliza el consumo en un rango seguro y constante ($O(1)$).

Repositorio de Github Grupal:

https://github.com/freddyordonez/Portafolio-de-Estructura-de-Datos.git

6. Resultados esperados:

- Los estudiantes lograrán observar de manera tangible cómo una referencia activa en una estructura de datos (Arreglo) impide la liberación de memoria dinámica (*Heap*). Aplicando un límite lógico, lograrán transformar un consumo de complejidad $S(n) = O(n)$ (infinito en el tiempo) a uno constante $S(n) = O(1)$, aplicando principios de diseño energéticamente eficientes.

7. Preguntas de Control:

Desde una perspectiva filosófica profunda y analítica, responda en su informe:

- **Causa Material:** Al declarar un objeto complejo dentro de una función, ¿en qué "lugar material" (Stack o Heap) se asienta su estructura principal, y en qué lugar se asienta el puntero que lo referencia?

  Cuando se declara un objeto complejo como un arreglo de datos o una estructura pesada dentro de una función, **su estructura principal**

y sus datos dinámicos se asientan físicamente en el **Heap**. Esto ocurre porque los objetos complejos poseen un tamaño dinámico, impredecible y variable que no puede calcularse estáticamente durante la compilación, por lo que requieren una región de memoria flexible administrada por el motor V8 de JavaScript y gestionada por el recolector de basura.

Por el contrario, el **puntero o variable** de referencia que apunta a dicho objeto se asienta en el **Stack** dentro del marco de ejecución de la función. Este puntero almacena únicamente una dirección de memoria estática que sirve como enlace lógico hacia la ubicación real de los datos en el Heap, permitiendo que el programa localice y manipule la estructura compleja mientras el contexto léxico esté activo.

- **Causa Formal:** El contexto global de JavaScript tiene una "forma" que abarca todo el ciclo de vida del programa. ¿De qué manera la estructura formal del alcance (*scope*) global actúa como una jaula que le prohíbe actuar al Recolector de Basura?

  El alcance global actúa como una jaula por su forma: es la raíz (*root*) desde la cual el algoritmo Mark-and-Sweep empieza a recorrer la memoria. Este algoritmo marca como “en uso” todo lo que se pueda alcanzar desde esa raíz y solo barre lo que queda inalcanzable. El scope global existe desde que arranca el programa hasta que termina, así que nunca se “cierra”. Por eso, todo lo que cuelga de él siempre es alcanzable.

  En nuestro código, **`historialSatelitalGlobal`** está declarado en ese ámbito. Mientras exista, cada **`paquete`** que se agrega con **`push()`** queda enlazado a la raíz (arreglo → paquete → **`payload)`** y el recolector lo marca como vivo, aunque ninguna función lo vuelva a consultar. Por eso, aun forzando **`global.gc()`** con **`--expose-gc`**, la RAM siguió subiendo de 81 MB a 386 MB: el recolector sí se ejecutó, pero no encontró nada inalcanzable para barrer.

  La diferencia con un ámbito local es de forma: cuando una función termina, su contexto léxico desaparece y los objetos que solo ella referenciaba quedan huérfanos. El scope global nunca termina, así

que esa liberación no ocurre. La restricción no es una prohibición explícita al recolector, sino una consecuencia de la estructura del alcance. Para liberar la memoria hay que romper la referencia desde dentro, como hicimos en **`lab03_solucion.js` con `LIMITE_HISTORIAL = 50` y `shift()`**, que desvincula al paquete más antiguo para que el GC pueda reclamarlo.

- **Causa Eficiente:** El algoritmo de *Mark-and-Sweep* actúa como un destructor activo. Sin embargo, en un *Memory Leak*, ¿quién o qué es el agente responsable (causa eficiente) que detiene la hoja de la guadaña del Recolector de Basura?

  El agente responsable de detener al Recolector de Basura es la referencia o enlace activo que nosotros como desarrolladores dejamos olvidado en el código. El algoritmo de Mark-and-Sweep funciona revisando qué objetos están conectados a la raíz del programa. Si nosotros mantenemos una referencia conectada por error o un mal diseño, el algoritmo marca ese objeto como en uso y le prohíbe al Recolector destruirlo. Por lo tanto, no es culpa del recolector, sino del puntero activo que protege al dato y provoca la fuga de memoria.

- **Causa Final:** Si el hardware permite almacenar Gigabytes de información temporal, ¿cuál es el propósito final (telos) de programar una destrucción intencional de los datos que nosotros mismos hemos creado con tanto esfuerzo computacional?

  El propósito final (telos) de destruir intencionalmente los datos es conservar únicamente la información que sigue siendo útil para el funcionamiento del programa. Aunque el hardware pueda almacenar gigabytes, su capacidad es finita y los datos temporales pueden perder su utilidad mucho antes de que se agote la memoria.

  El esfuerzo computacional de crear un dato se justifica por la función que cumple, no por conservarlo indefinidamente. Por ejemplo, un programa puede generar miles de resultados intermedios para procesar una imagen; una vez obtenida la imagen final, muchos de esos resultados ya no son necesarios. Liberar su memoria permite atender nuevas tareas y evita acumular información obsoleta. Además,

eliminar datos sensibles cuando dejan de necesitarse reduce su exposición.

Desde la causa final, se destruyen datos para preservar la capacidad del sistema de seguir cumpliendo su propósito. El criterio es su utilidad futura frente al costo de conservarlos o volver a calcularlos. Por eso, una buena gestión elimina lo prescindible y conserva aquello que sería necesario o costoso recuperar.

- **Esencia y Existencia:** Si un paquete de coordenadas queda "olvidado" en el último índice de un arreglo sin que ninguna función lo vuelva a consultar jamás, físicamente ocupa electrones en la RAM, pero lógicamente es inaccesible. ¿Existe este dato en la realidad del software o ha perdido su esencia al perder su utilidad? Argumente.

  Existe físicamente, pero ya perdió su esencia. En el software hay dos formas de “existir”: la física, que es ocupar espacio en la RAM, y la lógica, que es tener un uso dentro del programa. Un paquete olvidado en el último índice cumple solo la primera. Nadie lo consulta, así que no aporta nada, como una caja guardada en una bodega a la que nadie va a volver. Por eso se dice que ese dato ya perdió su esencia, que era ser útil.

  Lo importante es que el Garbage Collector no lo sabe. Él solo revisa si el dato todavía está enlazado desde una referencia activa, y no si alguien lo va a usar. Como el arreglo sigue apuntando al paquete, el GC lo considera “en uso” y no lo borra. Eso fue lo que pasó en el trabajo realizado con historialSatelitalGlobal: aunque se forzó global.gc(), la memoria siguió subiendo hasta los 386 MB. Un dato así no está vivo, pero tampoco se puede liberar, y eso es una fuga de memoria. La solución es del programador, que debe soltar la referencia o limitar el historial, como se hizo con el máximo de 50 registros.

Repositorio de Github Grupal:

https://github.com/freddyordonez/Portafolio-de-Estructura-de-Datos.git

8. Bibliografía

- P. W. Bible and L. Moser, *An Open Guide to Data Structures and Algorithms*. PALNI Open Press, 2023.
  .Available: https://open.palni.org/
- Mozilla Developer Network, "Memory management and Garbage Collection in JavaScript", *MDN Web Docs*, 2023.
  Available:
  https://developer.mozilla.org/en-US/docs/Web/JavaScript/Memory_Management
- Node.js Foundation, "V8 JavaScript engine and CLI options", *Node.js API*, 2023.
  Available: https://nodejs.org/docs/latest-v20.x/api/cli.html